MixBytes()

OKX DEX-Router Security Audit Report

MARCH 13, 2026

Table of Contents

1. Introduction	**3**
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Assessment Methodology	8
1.5 Risk Classification	10
1.6 Summary of Findings	11
2. Findings Report	**13**
2.1 Critical	13
2.2 High	13
2.3 Medium	13
M-1 totalWeight Validation Allows Incomplete Routing	13
M-2 Reentrancy Allows Referrer to Drain User Funds in swapWrap	15
M-3 Double-Spend via Receiver Balance Manipulation in Final Slippage Check	17
2.4 Low	19
L-1 Permit2 Mode Silently Skips Token Transfer	19
L-2 Uniswap V3 Callback Writes to Solidity Free Memory Pointer	20
L-3 Typo in Constant Name	21
L-4 Weight Validation Logic Is Coupled to Loop Control Flow	22
L-5 Commission Recipient Addresses Are Not Validated for Zero Address	23
L-6 Uniswap V3 Swap Paths Do Not Explicitly Reject Fee-on-Transfer Tokens	24
L-7 _exeHop Assumes Non-Empty hops Array Without Validation	25
L-8 Revert Message Does Not Match Validated Condition	26
L-9 ETH Sent Directly to WNativeRelayer Cannot Be Recovered	27
L-10 Missing toToken Consistency Validation	28
L-11 Rounding Remainders Are Lost During Forked Amount Distribution	29

L-12 Malformed ABI Encoding of Revert Reasons Prevents Error Decoding 30

L-13 _exeForks Does Not Validate Consistency of Related Array Lengths 32

L-14 Referrer Can Block Swaps via Unbounded Gas Consumption 33

L-15 Missing msg.value Validation in Payable Swap Functions 34

L-16 Cross-Contract Reentrancy via Direct Adapter Transfers in mixAdapters 36

L-17 Reentrancy Guard Misconfiguration Causes Wrapper Reverts 41

3. About MixBytes 42

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Protocol Overview

OKX DEX Router EVM V1 is a sophisticated multi-protocol DEX aggregator designed to optimize token swaps across multiple decentralized exchanges in a single transaction. The protocol supports various swap mechanisms including standard Uniswap V2/V3 routes, DAG-based (Directed Acyclic Graph) routing for complex multi-path swaps, and PMM (Proactive Market Maker) integrations. It features a commission system for referrers, a trim mechanism for handling positive slippage, and a two-layer token approval architecture (TokenApprove and TokenApproveProxy) for secure fund management.

Audit Scope and Methodology

The security assessment was conducted by a team of 3 senior security auditors. The methodology employed included: comprehensive manual code review with focus on complex assembly blocks and cross-contract interactions, architectural analysis of the DAG routing and commission systems, static analysis using Slither and custom detection rules, and development of Proof-of-Concept exploits using Foundry to validate critical findings. Collaborative auditing sessions were conducted for the most complex code sections.

Attack Vectors Analyzed

In addition to our internal security checklists, we paid special attention to the following attack vectors specific to DEX aggregator architecture:

- **Reentrancy attacks** through ETH transfers, ERC-777 transfers and external adapter callbacks
- **Cross-contract reentrancy** via adapter integrations and callback mechanisms
- **Slippage manipulation** and sandwich attacks on swap execution
- **Balance manipulation** attacks exploiting balance-based slippage checks
- **Integer overflow/underflow** in weight calculations and token distributions
- **Access control bypass** in the token approval proxy system
- **Front-running and MEV** exploitation opportunities
- **Malformed calldata** and input validation edge cases
- **Upgradeable contract vulnerabilities** including initialization attacks

Out of Scope

The following items were explicitly excluded from this audit:

- Frontend application security and off-chain infrastructure
- Third-party adapter contracts and external DEX integrations
- Economic modeling and tokenomics analysis
- Gas optimization beyond security-relevant inefficiencies

Notable Observations

Beyond the formal findings, the audit team identified the following architectural considerations that users and integrators should be aware of:

1. **Packed Parameters and UI Visibility:** The protocol extensively uses packed parameters (e.g., rawData encoding pool addresses, weights, and flags in a single uint256). When users sign transactions via wallet interfaces, these packed values are not human-readable. In the event of a frontend compromise, malicious parameters could be injected without users being able to detect the manipulation through standard wallet UI. Users should verify transactions through independent means when dealing with significant amounts.

2. **Poisoned Tokens and Adapters:** While the router itself is secure against the analyzed attack vectors, token theft remains possible if users explicitly include malicious (poisoned) tokens or compromised addresses in their transaction calldata. The router executes whatever adapters and tokens are specified in the calldata without maintaining an on-chain whitelist. Users must ensure they interact only with verified token addresses and trusted adapter contracts.

3. **Centralization Risk in Token Approval System:** While DexRouter is completely stateless and holds no user funds between transactions, the TokenApproveProxy contract is Ownable. Users grant token approvals to TokenApprove, which only accepts calls from TokenApproveProxy. The owner of TokenApproveProxy can add arbitrary addresses to the allowedApprove mapping, potentially enabling unauthorized token transfers. Users should be aware of this trust assumption and monitor ownership changes. Consider implementing a timelock or multisig for owner operations to mitigate this centralization risk.

Conclusion and Recommendations

The codebase demonstrates sophisticated optimization techniques, particularly in assembly-level implementations for gas efficiency. However, the aggressive use of callbacks and direct transfers to external contracts introduces significant reentrancy risks that require immediate attention.

The overall code quality is good with well-structured architecture. After addressing the findings, the protocol will be significantly more secure for production deployment.

1.3 Project Overview

Summary

Title	Description
Client	OKX
Category	DEX
Project	DEX-Router
Type	Solidity
Platform	EVM
Timeline	10.11.2025-13.03.2026

Scope of Audit

File	Link
contracts/8/libraries/Address.sol	contracts/8/libraries/Address.sol
contracts/8/libraries/CommissionLib.sol	contracts/8/libraries/CommissionLib.sol
contracts/8/libraries/CommonLib.sol	contracts/8/libraries/CommonLib.sol
contracts/8/libraries/CommonUtils.sol	contracts/8/libraries/CommonUtils.sol
contracts/8/libraries/CustomRevert.sol	contracts/8/libraries/CustomRevert.sol
contracts/8/libraries/EthReceiver.sol	contracts/8/libraries/EthReceiver.sol
contracts/8/libraries/PMMLib.sol	contracts/8/libraries/PMMLib.sol
contracts/8/libraries/Permitable.sol	contracts/8/libraries/Permitable.sol
contracts/8/libraries/ RevertReasonForwarder.sol	contracts/8/libraries/ RevertReasonForwarder.sol
contracts/8/libraries/ RevertReasonParser.sol	contracts/8/libraries/ RevertReasonParser.sol
contracts/8/libraries/SafeCast.sol	contracts/8/libraries/SafeCast.sol

File	Link
contracts/8/libraries/SafeERC20.sol	contracts/8/libraries/SafeERC20.sol
contracts/8/libraries/SafeMath.sol	contracts/8/libraries/SafeMath.sol
contracts/8/libraries/TempStorage.sol	contracts/8/libraries/TempStorage.sol
contracts/8/libraries/UniswapTokenInfoHelper.sol	contracts/8/libraries/UniswapTokenInfoHelper.sol
contracts/8/libraries/UniversalERC20.sol	contracts/8/libraries/UniversalERC20.sol
contracts/8/utils/WNativeRelayer.sol	contracts/8/utils/WNativeRelayer.sol
contracts/8/storage/PMMRouterStorage.sol	contracts/8/storage/PMMRouterStorage.sol
contracts/8/storage/DexRouterStorage.sol	contracts/8/storage/DexRouterStorage.sol
contracts/8/TokenApproveProxy.sol	contracts/8/TokenApproveProxy.sol
contracts/8/TokenApprove.sol	contracts/8/TokenApprove.sol
contracts/8/UnxswapRouter.sol	contracts/8/UnxswapRouter.sol
contracts/8/UnxswapV3Router.sol	contracts/8/UnxswapV3Router.sol
contracts/8/DexRouter.sol	contracts/8/DexRouter.sol
contracts/8/DagRouter.sol	contracts/8/DagRouter.sol

Versions Log

Date	Commit Hash	Note
10.11.2025	ad4c5ed1099bb9426c9615558ce47ab9982a485d	Initial Commit
26.11.2025	ce3446eb1183299904b131cb10925a3719c4bcbc	Commit With Tests
16.02.2026	d9f97c72324a20851aa2647815e9a792eed5992c	Reaudit Commit
03.03.2026	2498b86a224932e70412324fd70b1c65b2ebf0fa	2nd Reaudit Commit

Date	Commit Hash	Note
13.03.2026	da81322161d486bcf7e3f0da27dbc73ede901050	Custom Errors Refactor Commit

Mainnet Deployments

File	Address	Blockchain
contracts/8/DexRouter.sol	0x22eef0C15678c482DcAC05c0d102363fc31f8C81	Ethereum

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	**Project Architecture Review:** • Review project documentation • Conduct a general code review • Perform reverse engineering to analyze the project's architecture based solely on the source code • Develop an independent perspective on the project's architecture • Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: • Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. • Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. • Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. • Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: • Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. • Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: · Cross-check findings among auditors · Discuss identified issues · Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	**Bug Fixing & Re-Audit:** · The client addresses the identified issues and provides feedback · Auditors verify the fixes and update their statuses with supporting evidence · A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	**Final Code Verification & Public Audit Report:** · Verify the final code version against recommendations and their statuses · Check deployed contracts for correct initialization parameters · Confirm that the deployed code matches the audited version · Issue a public audit report, published on our official GitHub repository · Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likehood: High	Critical	High	Medium
Likehood: Medium	High	Medium	Low
Likehood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50-60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	3
Low	17

Findings Statuses

ID	Finding	Severity	Status
M-1	totalWeight Validation Allows Incomplete Routing	Medium	Fixed
M-2	Reentrancy Allows Referrer to Drain User Funds in swapWrap	Medium	Fixed
M-3	Double-Spend via Receiver Balance Manipulation in Final Slippage Check	Medium	Acknowledged
L-1	Permit2 Mode Silently Skips Token Transfer	Low	Fixed
L-2	Uniswap V3 Callback Writes to Solidity Free Memory Pointer	Low	Fixed
L-3	Typo in Constant Name	Low	Fixed
L-4	Weight Validation Logic Is Coupled to Loop Control Flow	Low	Fixed
L-5	Commission Recipient Addresses Are Not Validated for Zero Address	Low	Fixed
L-6	Uniswap V3 Swap Paths Do Not Explicitly Reject Fee-on-Transfer Tokens	Low	Acknowledged
L-7	_exeHop Assumes Non-Empty hops Array Without Validation	Low	Acknowledged
L-8	Revert Message Does Not Match Validated Condition	Low	Fixed

L-9	ETH Sent Directly to WNativeRelayer Cannot Be Recovered	Low	Acknowledged
L-10	Missing toToken Consistency Validation	Low	Acknowledged
L-11	Rounding Remainders Are Lost During Forked Amount Distribution	Low	Fixed
L-12	Malformed ABI Encoding of Revert Reasons Prevents Error Decoding	Low	Acknowledged
L-13	_exeForks Does Not Validate Consistency of Related Array Lengths	Low	Acknowledged
L-14	Referrer Can Block Swaps via Unbounded Gas Consumption	Low	Fixed
L-15	Missing msg.value Validation in Payable Swap Functions	Low	Fixed
L-16	Cross-Contract Reentrancy via Direct Adapter Transfers in mixAdapters	Low	Fixed
L-17	Reentrancy Guard Misconfiguration Causes Wrapper Reverts	Low	Fixed

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

M-1	totalWeight Validation Allows Incomplete Routing		
Severity	Medium	**Status**	Fixed in d9f97c72

Description

The issue was identified in the `_exeForks` function of the `DexRouter` contract.
The function validates `totalWeight` using `<= 10_000` instead of requiring it to equal `10_000`. This allows routing configurations where part of the input amount is not assigned to any adapter and remains in the router.
In contrast, the `_exeNode` function in `DagRouter` correctly enforces `totalWeight == 10_000`.
This issue is classified as **Medium** severity because it can cause user funds to remain stuck in the router and result in users receiving less output than expected.

Recommendation

We recommend changing the validation to require `totalWeight == 10_000` and enforcing that `totalBatchAmount` equals `_baseRequest.fromTokenAmount`.

Client's Commentary:

Client:

Request for severity downgrade.
Impact: This issue results from routes constructed with totalWeight < 10_000, which can cause incomplete allocation of the intended input across adapters and therefore lower-than-expected output. Any undistributed remainder stays with the effective payer (either the user for the first hop, or the router for subsequent hops), and is not a systematic protocol-wide drain. We assess the realistic impact as Low/Medium (transaction-scoped degradation / potential residue when the router is the payer).

- *Note: For swap flows that enforce minReturn, incomplete routing (lower output) will typically cause the transaction to revert rather than silently succeed; therefore the primary risk is misconfiguration/DoS and potential residue only in cases where the router is the payer or where no minReturn check is applied.*
  Likelihood: Exploitation requires malformed or misconfigured routing parameters (weights not summing to 10_000). Under standard order-construction flows, such routes should be rejected/avoided. We therefore assess likelihood as Low.
  Recommended adjustment: Low (Low Likelihood × Low Impact)

MixBytes:

*This assessment is based on a security model, where a realistic on-chain configuration that can produce a valid non-reverting transaction outcome while causing partial or unintended loss of user funds, is considered at least a **Medium** severity.*

*In this case, under-allocation of `totalWeight` can lead to partial loss of user funds while the transaction succeeds on-chain. While a `minReturn` check is present, it is not a reliable safeguard in this context. The unallocated portion may fall within the allowed slippage bounds, allowing the transaction to pass the check while still producing a lower-than-expected result. Such loss directly corresponds to **Medium** impact, as it affects user assets rather than causing a pure denial of service.*

*The likelihood is also assessed as **Medium**, since routing parameters are commonly constructed off-chain and rely on frontend logic, quoting, and aggregation. Frontend or integration errors, edge cases, or future UI changes can realistically produce such routes even if the current UI avoids them.*

*Given that, we keep the issue classified as **Medium** severity.*

Client:

Accept.

M-2	Reentrancy Allows Referrer to Drain User Funds in swapWrap		
Severity	Medium	**Status**	Fixed in d9f97c72

Description

The issue was identified in the swapWrap flow of the DexRouter contract, specifically in the _doCommissionFromToken function.

When fromToken is ETH, _doCommissionFromToken performs an external call to the referrer by sending ETH. This call can trigger a fallback or receive function on the referrer contract before the swap execution completes, enabling a reentrancy attack.

Because the router captures balanceBefore prior to the external call, a malicious referrer can manipulate the router's token balances during reentrancy. This allows the attacker to interfere with the subsequent balance-delta-based accounting used to determine the actual swap output.

A concrete attack path for swapWrap is as follows:

1. The attacker deploys and promotes a malicious referrer contract.
2. A victim submits a swapWrap transaction swapping 100 ETH to WETH with a fromToken commission and trimming enabled, setting the referrer to the malicious contract.
3. The attacker front-runs the transaction by sending 100 WETH to the router, causing the router to temporarily hold both 100 ETH and 100 WETH.
4. During execution, _doCommissionFromToken records balanceBefore as 100 WETH.
5. The router sends ETH to the referrer, triggering the referrer's receive() function.
6. Inside receive(), the malicious referrer reenters the router and executes smartSwapByInvest, swapping ETH to WETH and sending the output to the attacker.
7. The router wraps ETH via IWETH.deposit, leaving the attacker with 100 WETH.
8. Execution resumes, and _doCommissionAndTrimToToken computes the input amount as balanceAfter - balanceBefore, which evaluates to zero.
9. As a result, zero tokens are sent to the intended receiver, while the attacker ends up with both 100 WETH and 100 ETH.

Since swapWrap does not enforce a slippage check, the attacker can drain the full user deposit. Similar reentrancy patterns affect other swap paths that combine commissions and trims; while those may be partially bounded by slippage checks, the attacker can still steal the difference between actual and maximum allowed slippage.

Recommendation

We recommend adding a nonReentrant modifier to all external router entry points and moving the balanceBefore calculation to occur strictly after _doCommissionFromToken completes.

Client's Commentary:

Client:

We fully acknowledge the theoretical existence of this vulnerability at the code level and are committed to implementing a code fix to address the root cause. Our primary goal in this dialogue is to align on a final severity rating that accurately reflects the real-world risk profile of this issue within our specific, live production environment, ensuring appropriate prioritization alongside other findings.

We understand your framework evaluates the "worst-case scenario" from a code perspective. However, industry best practices in risk assessment often incorporate the effectiveness of existing, deployed mitigations when determining the final severity. Based on this principle, we would like to provide further context for a joint reassessment.

*Regarding **Impact**: Theoretical Capability vs. Exploitable Scope*

We agree that in a pure, laboratory-style test environment without constraints, the vulnerability could theoretically affect a user's funds within a single transaction.

However, the exploitable impact in our production ecosystem is significantly mitigated by established, non-bypassable business logic controls:

- *The attack path is strictly dependent on a specific, non-standard set of parameters (e.g., particular trim/commission settings and an attacker-controlled referrer).*
- *In our live system, the official front-end and integrated SDKs enforce strict validation and limits on these parameters, making it impossible for standard user interactions to produce the malicious parameter combination required to trigger the vulnerable settlement path.*
- *Consequently, under standard user interaction flows, the "theoretical impact" is unreachable. The practical, exploitable impact is confined to an edge case involving sophisticated users interacting directly with the router via custom contracts while deliberately choosing malicious parameters.*
- *In addition, if "single-transaction fund loss" is always escalated to High/Critical in the matrix, the differentiation between severity levels becomes compressed, leaving limited room to further escalate more destructive vulnerabilities (e.g., those that can systematically impact protocol funds / batch user funds, or enable sustainable accumulative theft paths). This reduces comparability and tiering value across different impact surfaces.*

*Therefore, we believe the actual impact severity is more accurately categorized as **Medium**, reflecting a flaw that is isolated to a very specific interaction pattern and does not threaten the core protocol or user funds at scale.*

*Regarding **Likelihood**: Code Accessibility vs. Real-World Exploitation Prerequisites*

While the code path may be "direct" in isolation, successful exploitation requires overcoming significant hurdles in practice:

- *High Likelihood Threshold Not Met: The requirement for precise condition alignment (specific callback, specific state) does not align well with the "50-60% annual probability" or "triggerable by any actor under normal conditions" descriptors typical of a High likelihood rating.*
- *Production Barriers are Integral to Risk Assessment: The need for an attacker to convince a user to abandon the official interface and interact with a malicious third-party dApp/contract substantially increases the attack's complexity and cost, reducing its probability. These deployed barriers are permanent features of our operational environment.*
- *Absence of Observed Exploits: The relevant contract logic has been public for a notable period. The lack of any observed exploitation attempts on-chain serves as a supporting data point for its currently low probability of occurrence.*

*Considering these environmental and operational factors, we propose the likelihood is more appropriately assessed as **Low**.*

Seeking Consensus on Final Severity

We respectfully request a final severity rating that balances theoretical soundness with practical context:

- *We commit unequivocally to fixing the code.*
- *We assert that current production safeguards significantly reduce the real-world risk.*
- *We propose a final severity of **Low/Medium**, based on Medium (Exploitable) Impact * Low Likelihood.*

This adjusted rating would:

1. *More transparently communicate the actual, contained risk level to stakeholders.*
2. *Allow for a risk-prioritized remediation schedule that focuses immediate resources on findings with higher actionable threat levels.*

MixBytes:

Upon further assessment, we agree to classify this issue as Medium severity based on a High Impact combined with Low Likelihood.

From an impact perspective, the vulnerability is capable of causing complete loss of user funds within a single transaction, which constitutes a high-severity outcome at the individual user level.

However, exploitation requires a narrowly constrained execution context, including explicit user authorization of the referrer and non-standard parameter combinations that are not part of typical production flows. These prerequisites materially reduce the probability of occurrence in practice, supporting a low likelihood classification.

While the severity is adjusted accordingly, the remediation is strongly recommended to prevent latent risk should execution assumptions change.

Client:

Accept.

M-3	Double-Spend via Receiver Balance Manipulation in Final Slippage Check		
Severity	Medium	**Status**	Acknowledged

Description

The issue was identified in the final slippage check logic of the DexRouter contract. The router verifies slippage by checking the final balance of toToken held by the receiver. However, this balance may increase for reasons unrelated to the current swap. If the receiver is also a maker in another protocol or has pending rfq orders, a referrer or maker can trigger those executions during a callback, increasing the receiver's toToken balance mid-swap. Because the router assumes that any balance increase originates solely from the current swap, any residue increases the slippage check tolerance, thereby allowing stealing arbitrary large amounts of toToken from the router via reentrancy or sandwich attack, limited only by the size of the external orders triggered during execution.

Recommendation

We recommend performing the final slippage check against balances held by the router itself and only transferring tokens to the receiver after the slippage condition has been fully validated.

Client's Commentary:

Client:

We acknowledge the theoretical existence of this issue at the code/mechanism level. We will carefully assess and implement appropriate defensive measures. Our primary goal in this dialogue is to align on a final severity rating that reflects the real-world risk profile of this issue within our specific, live production environment.We understand your framework evaluates the "worst-case scenario" from a code perspective. However, industry best practices in risk assessment often incorporate the effectiveness of existing, deployed mitigations when determining the final severity. Based on this principle, we would like to provide further context for a joint reassessment.

*Regarding **Impact**: Theoretical Capability vs. Exploitable Scope*

*We agree that, in a fully unconstrained test setting, receiver balance manipulation could theoretically weaken the intended protection of the final slippage check within a single transaction.However, the exploitable scope remains transaction-scoped. We understand and note that your matrix definition includes "loss of >1% of interacting user funds." Our concern is that directly equating "single-transaction user fund loss" with "protocol-level high impact" broadens the applicability of High Impact, which differs from our consistent understanding of the original definition (Impact being tied to protocol-level risk). Therefore, consistent with our position in C1, we believe the practical impact is more accurately categorized as **Medium** (transaction-scoped / single-user-level impact).*

*Regarding **Likelihood**: Code Accessibility vs. Real-World Exploitation Prerequisites*

While the mechanism may appear "simple" in isolation, successful exploitation requires overcoming significant practical hurdles that materially reduce probability:

- *High Likelihood threshold not met (per your matrix): High requires a "50–60% probability of occurring within a year" and to be "triggerable by any actor under normal conditions." This scenario depends on precise preconditions and timing alignment (external credit-in must occur during the swap execution window and in the same toToken), which does not match a broadly likely market condition that an actor cannot influence. We do not have objective evidence supporting a 50–60% annual probability.*
- *Condition (1) is not a common user state: even under the abstraction that "only two conditions are needed," condition (1)—the receiver having a publicly triggerable toToken credit-in in an external protocol (e.g., unclaimed balances or third-party-executable orders/settlements) that can be realized within the same execution window—is not a typical state for most ordinary users and has additional constraints on "same token + triggerable timing."*
- *Production barriers are integral to risk assessment: we believe deployed, production-side controls should be incorporated into Likelihood. In our current production environment, referrer sourcing and usage scope are strictly constrained (e.g., only permitted for specific business identities/sources), and features such as trim are enabled only in specific business scenarios (e.g., toB*

users). These operational constraints reduce the reachability of the generic trigger conditions described in the report through standard user flows.

- *Absence of observed exploits: the contract is open-source and has been public for a notable period. To date, we have not observed real attacks exploiting this path in production (this does not prove absence of risk, but serves as a supporting data point for currently low observed incidence).*

*Considering these environmental and operational factors, we propose the likelihood is more appropriately assessed as **Low**.*

Seeking Consensus on Final Severity

We respectfully request a final severity rating that balances theoretical soundness with practical context:

- *We will implement appropriate defensive measures.*
- *We assert that current production safeguards significantly reduce the real-world risk.*
- *We propose a final severity of **Low/Medium**, based on Medium (Exploitable) Impact + Low Likelihood.*

We also note that Uniswap Labs has classified similar "receiver balance-based accounting can be polluted by external credit-ins" issues as High severity. Under our production constraints and more complex triggering conditions, we believe the appropriate severity should be no higher than High, and we request a downgrade from Critical accordingly.

MixBytes:

We agree to classify this issue as Medium severity, based on a High Impact combined with Low Likelihood.

From an impact perspective, the identified weakness in balance-based settlement logic can, under specific conditions, result in significant unintended value transfer during a single transaction, potentially leading to near-complete loss of the user's expected swap outcome. Such an outcome represents a high-impact scenario at the individual user level.

However, successful exploitation depends on a narrowly defined set of prerequisites, including the presence of externally triggerable balance inflows for the receiver in the same asset and precise execution timing within the swap lifecycle. These conditions are not representative of typical user states or standard execution paths and materially reduce the probability of occurrence in practice.

Client:

Accept. We added nonReentrant protections on key external router entry points to reduce callback-driven reentrancy surfaces.

2.4 Low

L-1	Permit2 Mode Silently Skips Token Transfer		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in the _transferInternal function of the CommonLib library.
When the transfer mode is set to _MODE_PERMIT2, the function exits early without performing a token transfer and without reverting. The caller receives no signal that the transfer did not occur, and the swap execution continues as if the transfer succeeded.

Recommendation

We recommend reverting with a descriptive error when _MODE_PERMIT2 is selected.

Client's Commentary:

Acknowledged.

L-2	Uniswap V3 Callback Writes to Solidity Free Memory Pointer		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in the uniswapV3SwapCallback function of the UnxswapV3Router contract. The callback uses inline assembly to store returned data at memory position 0x40, which Solidity reserves as the free memory pointer. Overwriting this slot alters Solidity's internal memory management.

While the current implementation does not perform further memory allocations after this write, the correctness of the code relies on this assumption. Any future modification that introduces Solidity memory usage after the callback may result in corrupted memory state.

Recommendation

We recommend writing callback data to a memory location that does not overlap with Solidity's free memory pointer.

Client's Commentary:

Client:

We acknowledge the theoretical concern raised here as a Solidity memory-convention / future-maintenance consideration. Our primary goal is to align on a disposition that accurately reflects the real-world security risk in our current production deployment, rather than treating a non-exploitable coding-style issue as a reportable vulnerability. We understand your framework evaluates worst-case scenarios from a code perspective. However, industry best practices also distinguish between (i) directly exploitable issues under supported flows, and (ii) defensive coding / maintainability considerations that may become relevant only if future refactors introduce additional memory allocations or attacker-controlled influence.

Regarding Impact: Theoretical Capability vs. Exploitable Scope

In the current implementation, there are no Solidity memory allocations performed after the referenced memory write, and the behavior does not provide an attacker-controlled primitive that can lead to fund loss. In addition, the callback invocations are clean and context-aware (i.e., the execution context is well-defined), and do not rely on the potentially "polluted" memory region. As such, this item is not a production-reachable security issue by itself under the present code and supported execution paths.

Request

Given the above, we respectfully request keeping this item as Informational (hardening) or removing it from the Findings section. We will fix it in the next update for clarity and maintainability.

MixBytes:

We agree this is a minor hardening / maintainability concern and does not represent an exploitable issue under the current implementation.

Please, also note that our report format does not have a separate Informational section. Findings that are non-mandatory to fix but would improve code safety/clarity are placed in the Low section as well. We will keep this item for completeness, so it's clear the team reviewed the concern, even if it does not require urgent remediation.

L-3	Typo in Constant Name		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in the UnxswapRouter contract.
The constant _WITHDRAW_TRNASFER_SELECTOR contains a spelling mistake, and the modifier isExpired is named in a way that contradicts its behavior (it reverts when the deadline *has already expired*).

Recommendation

We recommend correcting the constant name and renaming the modifier to better reflect its logic.

Client's Commentary:

Client:

We acknowledge this item as a naming/readability issue rather than a correctness or security concern. Our primary goal is to keep the Findings list focused on security-relevant issues under supported execution paths.
Regarding Impact: Theoretical Capability vs. Exploitable Scope
Naming typos do not impact contract correctness or security properties and do not provide an attacker-controlled primitive. This is strictly a readability/maintainability improvement.
Request
We respectfully request removing this item from the Findings list.
Fix Status
We will fix the naming typos in the next update for clarity and maintainability.

MixBytes:

We agree this is a readability item rather than a security issue under the intended execution model.
We will keep this item for completeness, so it's clear the team reviewed the concern, even if it does not require urgent remediation.

L-4	Weight Validation Logic Is Coupled to Loop Control Flow		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in the `_exeForks` function of the `DexRouter` contract and the `_exeNode` function of the `DagRouter` contract.
The validation that checks whether `totalWeight` equals `10_000` is implemented as a conditional inside the loop and is only triggered on the final iteration.

Recommendation

We recommend performing the `totalWeight` validation once after the loop completes.

Client's Commentary:

Client:

We acknowledge this item as a refactor/readability recommendation rather than a security-relevant finding. Our primary goal is to keep the Findings list focused on issues that materially affect correctness, enforced invariants, or exploitability under supported execution paths.
Regarding Impact: Theoretical Capability vs. Exploitable Scope
This recommendation does not change or weaken the enforced invariant itself, nor does it introduce or remove any attacker-controlled primitive. It is primarily a code-structure/readability improvement. We also note that this refactor was suggested during our internal audit/review process and preserves the existing requirement that the weights satisfy the expected constraints, while helping reduce gas usage.
Request
We respectfully request removing this item from the Findings list.

MixBytes:

We agree this is a refactor item rather than a security issue under the intended execution model.
We will keep this item for completeness, so it's clear the team reviewed the concern, even if it doesn't require the change in the current design.

L-5	Commission Recipient Addresses Are Not Validated for Zero Address		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in the `_validateCommissionInfo` function of the `CommissionLib` library. The function validates commission configuration parameters but does not verify that referrers, trim charge, or trim receiver addresses are non-zero. If any of these addresses are set to `address(0)`, commission or trim amounts will be transferred to the zero address and irreversibly burned.

Recommendation

We recommend validating that all commission-related recipient addresses are non-zero before executing transfers.

Client's Commentary:

Client:

We acknowledge this item as primarily a hardening/defense-in-depth consideration rather than a production-reachable security issue under our supported configuration model. Our goal is to keep the Findings list focused on issues that materially affect exploitability in supported flows.

Regarding Impact: Theoretical Capability vs. Exploitable Scope

Commission/trim recipient addresses are controlled by our order-construction and configuration processes. Under these supported assumptions, this does not introduce an attacker-controlled primitive.

Regarding Remediation Practicality

Adding multiple runtime checks would increase gas costs and code complexity across frequently used paths. If additional hardening is desired, we are open to considering a minimal, targeted validation set that provides meaningful protection with limited overhead.

Request

We respectfully request removing it from the Findings section.

MixBytes:

We agree this is a defense-in-depth validation item rather than a security issue under the intended execution model.

We will keep this item for completeness, so it's clear the team reviewed the concern, even if it doesn't require the change in the current design.

L-6	Uniswap V3 Swap Paths Do Not Explicitly Reject Fee-on-Transfer Tokens		
Severity	Low	**Status**	Acknowledged

Description

The issue was identified in the Uniswap V3 swap execution logic of the UnxswapV3Router contract. Uniswap V3 pools inherently do not support fee-on-transfer (tax) tokens and will revert when such tokens are used. The router does not explicitly detect or document this limitation, which may lead developers or integrators to assume that fee-on-transfer tokens are supported consistently across all swap paths.

Recommendation

We recommend explicitly documenting this limitation or adding a guard that rejects fee-on-transfer tokens in Uniswap V3 swap paths.

Client's Commentary:

Client: *Reject. Uniswap V3 pools do not support fee-on-transfer (tax) tokens by design and will revert when such tokens are used.*

MixBytes: *This is a minor issue related to an implicit Uniswap V3 limitation that may cause non-obvious reverts. Marked as acknowledged.*

Client:

We respectfully request removing it from the Findings section.

MixBytes: *We will keep this item for completeness, so it's clear the team reviewed the concern, even if it doesn't require the change in the current design.*

L-7	_exeHop Assumes Non-Empty hops Array Without Validation		
Severity	Low	Status	Acknowledged

Description

The issue was identified in the _exeHop function of the DexRouter contract.
The function assumes that the hops array contains at least one element and uses it without validating this assumption. If an empty array is passed, execution may revert at an unrelated location or follow an unintended code path.

Recommendation

We recommend explicitly validating that hops.length > 0 at the beginning of the function.

Client's Commentary:

Client:

We acknowledge the concern being raised, but under the current smartSwap call flow we do not believe this represents a production-reachable issue.
Regarding Impact: Theoretical Capability vs. Exploitable Scope
Under the current implementation, malformed routes with empty hops are expected to revert earlier due to bounds checks implied by array indexing (e.g., accessing batches[0][0] / batches[i][0]), rather than reaching _exeHop with an empty hops array.
Request
Given the above, we respectfully request removing this item from the Findings list.

MixBytes:

We agree this is a defense-in-depth / route-sanity validation item rather than a security issue under the intended execution model. We will keep this item for completeness, so it's clear the team reviewed the concern, even if it doesn't require the change in the current design.

L-8	Revert Message Does Not Match Validated Condition		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in the _smartSwapInternal function of the DexRouter contract.
The revert message refers to the "number of batches", while the actual check validates the total batch *amount*. This mismatch can mislead developers when diagnosing configuration or input errors.

Recommendation

We recommend updating the revert message to accurately reflect the validated condition.

Client's Commentary:

Accept.

L-9	ETH Sent Directly to WNativeRelayer Cannot Be Recovered		
Severity	Low	**Status**	Acknowledged

Description

The issue was identified in the WNativeRelayer contract.
The contract accepts ETH via its receive() function but does not provide any mechanism to withdraw ETH sent directly to the contract. Only ETH obtained by unwrapping WETH through the withdraw function can be recovered.
Any ETH transferred directly to the contract address becomes permanently locked.

Recommendation

We recommend explicitly restricting the receive() function so that only the WETH contract can send ETH to WNativeRelayer or adding a method to withdraw the contract's ETH balance.

Client's Commentary:

Client: *Reject. By design. WNativeRelayer is intended to act as a WETH unwrap relay for whitelisted callers.*

MixBytes: *Acknowledged. We updated the recommendation to reflect the intended unwrap-relay design and address the reported concern.*

Client: *We respectfully request removing this item from the Findings list as by design: Restricting receive() to accept ETH only from WETH would reduce accidental direct transfers, but it cannot fully prevent forced ETH transfers (e.g., via selfdestruct). Adding a generic ETH withdrawal method would introduce a custody-like control surface and is not aligned with our non-custodial design assumptions; therefore, we prefer to avoid introducing privileged withdrawal mechanics.*

MixBytes:

We agree this is a minor, design-level edge case rather than a security vulnerability.
We will keep it in the report for completeness, so it's clear the team reviewed and acknowledged the behavior, even if it doesn't require the change in the current design.

L-10	Missing toToken Consistency Validation		
Severity	Low	Status	Acknowledged

Description

`_baseRequest.toToken` is not validated against the final output tokens of all terminal nodes in DAG and smart swap paths. A misconfigured route can therefore result in output tokens that are not forwarded to the user.

Recommendation

We recommend validating that `_baseRequest.toToken` matches the final token of every terminal swap path.

Client's Commentary:

Client:

Reject.

Route parameters (including _baseRequest.toToken and the output token of each terminal path) are submitted by the transaction initiator. The router is a permissionless execution layer and does not provide semantic correctness guarantees for arbitrarily user-crafted routes. If a user or third-party integrator misconfigures the route such that _baseRequest.toToken does not match the terminal output token, the resulting funds not being forwarded is a caller-side configuration/integration error, and the responsibility lies with the route builder.

The router performs a final minReturn check and will revert the entire transaction if the requested _baseRequest.toToken is not received in sufficient amount. Therefore, a misconfigured route typically results in a revert rather than silently forwarding an unexpected token or permanently losing outputs (state changes are reverted atomically).

We consider it a defense-in-depth validation improvement rather than a standalone theft vector under the current minReturn checks.

MixBytes:

*Given that this issue only arises from route builder misconfiguration and typically results in a revert due to `minReturn` checks, we lower the severity to **Low**. Marked as acknowledged.*

Client:

We acknowledge this behavior as by design and not a security vulnerability under the router's intended execution model.

Regarding Design Intent / Security Guarantees

The router is a permissionless execution engine and guarantees settlement only against the user-specified minReturn for _baseRequest.toToken (otherwise the transaction reverts). Under atomic execution plus minReturn checks, the router does not need to enforce full terminal-path token consistency beyond the declared toToken.

Regarding Remediation Practicality

Enforcing full terminal-path token consistency would be a semantic constraint for the route builder rather than a safety requirement of the execution engine. It would also add gas costs and implementation complexity, especially for DAG and smart-swap routes, without providing additional security guarantees under the current model.

Request

Given the above, we respectfully request removing this item from the Findings list.

MixBytes:

We agree this is a defense-in-depth / route-sanity validation item rather than a security issue under the intended execution model. We will keep it in the report for completeness, so it's clear the team reviewed and acknowledged the behavior, even if it doesn't require the change in the current design.

L-11	Rounding Remainders Are Lost During Forked Amount Distribution		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in the _exeForks function of the DexRouter contract.
When distributing token amounts across multiple adapters, integer division is used to compute per-adapter amounts. Any dust remainder caused by rounding is not assigned to any adapter, remaining unaccounted for.
Unlike _exeNode in DagRouter, there is no logic that forwards the remainder to the final adapter.

Recommendation

We recommend assigning any remaining amount to the last adapter to ensure full distribution.

Client's Commentary:

Acknowledged.

L-12	Malformed ABI Encoding of Revert Reasons Prevents Error Decoding		
Severity	Low	**Status**	Acknowledged

Description

In several locations, the encoded revert reason data contains incorrect string length prefixes or typographical errors, which makes the revert reason impossible to ABI-decode by standard tooling (ethers.js, Foundry, Hardhat).
Concrete instances include:

- `UnxswapRouter.sol` **– empty pools revert**

  The revert reason length prefix is incorrect. The encoded length does not match the actual string size, causing decoding to fail:

  ```
  revertWithReason(
      0x000000b656d70747920706f6f6c730000000000000000000000000000000000000000,
      0x4e
  );
  ```

  The prefix `0x000000b6` is invalid and should encode the actual byte length of `"empty pools"`.

- `UnxswapRouter.sol` **– "token0 balance failed"**

  The provided length parameter does not match the encoded string length:

  ```
  revertWithReason(
      0x00000015746f6b656e302062616c616e6365206661696c656400000000000000000,
      0x56  // Incorrect length
  );
  ```

- `UnxswapV3Router.sol` **– "get token1 failed"**

  The revert reason length is off by one byte:

  ```
  _revertWithReason(
      0x0000001167657420746f6b656e31206661696c656400000000000000000000000000,
      84  // Should be 85
  );
  ```

- `CommissionLib.sol` **– typo in revert string**

  The revert reason contains a spelling mistake (`referer` instead of `referrer`), which does not match the documented error message:

```
_revertWithReason(
    0x0000001b7472616e7366657220746f6b656e207265666572657220666169 6c00,
    0x5f
);
```

In all cases, the revert itself works correctly, but the revert reason cannot be decoded, making debugging, monitoring, and incident analysis significantly harder.

Recommendation

We recommend correcting the ABI length prefixes for all manually encoded revert reasons and fixing typographical errors in revert strings to ensure they can be decoded by standard tooling.

Client's Commentary:

Client:

We acknowledge this as an observability / developer-experience concern rather than a security issue.

Regarding Impact: Theoretical Capability vs. Exploitable Scope

This affects error decodeability and debugging ergonomics only. It does not change the underlying revert behavior, does not weaken any enforced invariant, and does not introduce an attacker-controlled primitive impacting fund safety. As such, it does not affect revert correctness or fund safety, but it can impact monitoring, incident triage, and tooling compatibility.

Design / Operational Trade-off

We keep the current custom revert encoding as an implementation trade-off. In our current operations, parts of our backend parsing/alerting logic rely on the existing revert byte patterns (including non-standard lengths/formatting and specific string variants). As a result, switching to fully standard ABI-encoded revert reasons may require coordinated backend/tooling updates and could otherwise temporarily reduce decodeability in our internal pipelines. That said, we acknowledge the value of standardizing revert encodings for broader tooling compatibility. We will evaluate this change and, where it can be adopted without negatively impacting production observability, plan a follow-up cleanup (e.g., staged backend updates and then standardization on-chain).

Request

Given the above, we respectfully request removing this item from the Findings list.

MixBytes:

We agree this is a minor observability / developer-experience issue.

It does not affect revert correctness, contract invariants, or fund safety. The impact is limited to standard tooling being unable to ABI-decode the revert reason strings (due to malformed length prefixes / typos), which can make debugging and monitoring less convenient.

We will keep it in the report for completeness, so it's clear the team reviewed and acknowledged the behavior, even if it doesn't require the change in the current design.

L-13	_exeForks Does Not Validate Consistency of Related Array Lengths		
Severity	Low	**Status**	Acknowledged

Description

The issue was identified in the _exeForks function of the DexRouter contract.
The function operates on several related arrays (mixAdapters, rawData, extraData, assetTo) and assumes that their lengths are consistent. This assumption is not validated. If the arrays differ in length, execution may revert unexpectedly or process mismatched data.

Recommendation

We recommend validating that all related arrays have equal lengths before executing forked swap logic.

Client's Commentary:

Client:

We acknowledge this as a sanity-check / defensive-programming improvement rather than a security-relevant vulnerability under supported execution paths.
Regarding Impact: Theoretical Capability vs. Exploitable Scope
This concerns input-shape validation (array-length consistency). Under our production flow, we enforce consistent array lengths during order construction; a mismatch represents a malformed call that will revert early or follow an unintended execution path, but does not by itself create an attacker-controlled primitive leading to fund loss.
Under the current implementation, malformed routes with empty hops are expected to revert earlier due to bounds checks implied by array indexing.
Request
Given the above, we respectfully request removing this item from the Findings list.

MixBytes:

We agree this is a minor defensive-programming issue and does not require mandatory changes.
We will keep it in the report for completeness, so it's clear the team reviewed and acknowledged the behavior, even if it doesn't require the change in the current design.

L-14	Referrer Can Block Swaps via Unbounded Gas Consumption		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in `_doCommissionFromTokenInternal` in `CommissionLib.sol`.
The function sends ETH to referrers using a low-level call with unlimited gas. A malicious referrer contract can revert or consume all gas, causing the entire swap transaction to revert.

Recommendation

We recommend preventing swap reverts when referrer payouts fail, for example by handling failures gracefully and enforcing a gas limit.

Client's Commentary:

Client:

Request for severity downgrade.

Rationale for downgrade:

Impact: This issue is transaction-scoped DoS (the swap reverts and no funds are lost). It does not lock funds on the contract nor enable theft. We therefore assess impact as Low Impact under the matrix.

*Likelihood: Requires users to actively select a malicious referrer → **Medium Likelihood**.*

Recommended adjustment: Low (Medium Likelihood × Low Impact)

MixBytes:

Given the transaction-scoped nature of the DoS, absence of direct fund loss, and the requirement for an explicitly chosen malicious referrer, we agree to lower the severity to Low.

Client:

Accept.

L-15	Missing msg.value Validation in Payable Swap Functions		
Severity	Low	Status	Fixed in d9f97c72

Description

The issue was identified in _dagSwapInternal and other payable functions in DagRouter.sol. When fromToken is ETH, the code wraps _baseRequest.fromTokenAmount into WETH but does not validate that msg.value matches this amount. Any excess ETH sent with the transaction is silently ignored and permanently lost.

This issue is classified as **Medium** severity because it can lead to silent loss of user funds due to incorrect ETH accounting.

Recommendation

We recommend validating that msg.value equals _baseRequest.fromTokenAmount plus any applicable commission amounts in all payable functions when fromToken equals ETH.

Client's Commentary:

Client:

Reject.

Rationale:

1. *Exact equality is not reliably achievable in our production flow. The required msg.value can differ by 1–2 wei due to off-chain quoting/rounding and multi-step aggregation, so enforcing msg.value == amount (or == amount + commissions) would introduce unnecessary false reverts.*
2. *Consistent with major DEX router designs (e.g., Uniswap / 1inch) .*
3. *Not a protocol-level risk under our model. The router is not intended to custody user funds; sending excess ETH is a user-side input mistake. This behavior is by design.*

MixBytes:

While strict `msg.value` equality may be impractical in aggregated routing flows, silently accepting excess ETH can still result in unintended user fund loss.

Notably, common router implementations mitigate this differently: Uniswap relies on multicall that allow explicit sweeping of residual ETH, while 1inch explicitly refunds any leftover ETH to the payer. In contrast, the current implementation does not provide any mechanism to recover excess ETH.

*Therefore, we keep the issue in the report as a **Medium**-severity accounting risk and suggest non-strict validation (`msg.value <= amount + EPS`) or explicit refunding of residual ETH as alternative mitigations.*

Client:

We acknowledge the theoretical existence of this issue at the integration level. Our primary goal in this dialogue is to align on a severity rating that accurately reflects the real-world risk profile of this behavior within our specific, live production environment and typical aggregator usage.We understand your framework evaluates "worst-case" behaviors from a code perspective. However, industry best practices in risk assessment also consider whether a scenario is realistically reachable under supported integrations, and whether it represents an exploitable protocol-level vulnerability versus an integration/sanity-check concern. Based on this principle, we would like to provide the following context for a joint reassessment.

*Regarding **Impact**: Theoretical Capability vs. Exploitable Scope*

*We view this as a sanity-check / integration-hardening item rather than a protocol-level vulnerability. The impact is strictly bounded to the caller's excess msg.value (i.e., ETH sent beyond what the route actually consumes). The scenario requires a misconstructed transaction where the caller overpays msg.value relative to the intended execution amount. In aggregated routing contexts, enforcing strict msg.value == required checks can be brittle and may introduce unnecessary false reverts. By contrast, explicitly refunding residual ETH achieves the same mitigation goal—preventing silent loss—while preserving compatibility with aggregator-style flows. Therefore, we believe the practical impact is most accurately categorized as **Low**.*

Regarding Likelihood: Code Accessibility vs. Real-World Exploitation Prerequisites

*Real-world manifestation primarily depends on upstream integration behavior (incorrect msg.value construction) rather than an adversary-driven exploit path. Under standard user interaction flows and well-formed integrations, the condition is not expected to occur frequently. Accordingly, we propose the likelihood is best assessed as **Low**.*

Request

*We respectfully request a severity downgrade to **Low**.*

Fix Status

This has been addressed by implementing explicit refunding of unused ETH via TransferLib.refundToken(_ETH, refundTo) in payable swap paths, ensuring that any msg.value in excess of the required amount is returned to the designated refund address.

MixBytes:

*We agree to downgrade this issue to **Low**.*

The only loss occurs if a caller sends excess msg.value due to an incorrectly constructed transaction. Since this router is intended to be used via an off-chain service that computes the required ETH amount, the likelihood of this happening in normal usage is low.

L-16	Cross-Contract Reentrancy via Direct Adapter Transfers in mixAdapters		
Severity	Low	**Status**	Fixed in d9f97c72

Description

The issue was identified in the smart swap execution flow of the DexRouter contract, specifically in the mixAdapters logic.

As an optimization, when only one downstream adapter exists, tokens are transferred directly from the current adapters to the next adapter, bypassing the router. This design introduces a cross-contract reentrancy risk.

For example, consider a swap path involving in the same *hop* Uniswap followed by a 1inch RFQ order that includes a maker callback. Uniswap transfers tokens directly to the next adapter. During the RFQ callback, the maker can reenter that adapter logic and siphon enough tokens to still satisfy the final slippage check. The result is that user funds are partially or fully stolen while the transaction completes successfully.

This issue is classified as **Low** severity because it enables reentrancy-based fund theft across adapter boundaries while bypassing internal accounting guarantees.

Recommendation

We recommend ensuring that the DexRouter always acts as the intermediate token holder between all swap steps and enforcing nonReentrant modifiers, rather than allowing direct adapter-to-adapter transfers.

Client's Commentary:

Client:

We fully acknowledge the theoretical existence of this issue at the mechanism level. We will carefully review the provided attack assumptions and, where applicable, consider targeted defense-in-depth measures to further harden the execution flow. Our primary goal in this dialogue is to align on a final severity rating that accurately reflects the real-world risk profile of this issue within our specific, live production environment. We understand your framework evaluates the "worst-case scenario" from a code perspective. However, industry best practices in risk assessment often incorporate the effectiveness of existing, deployed mitigations and operational constraints when determining the final severity. Based on this principle, we would like to provide further context for a joint reassessment.

*Regarding **Impact**: Theoretical Capability vs. Exploitable Scope*

*We agree that, in a fully unconstrained test setting, this design can enable third parties to capture value during cross-contract callbacks and impact the effective outcome of a swap.However, the exploitable loss is bounded by the slippage constraint: user loss manifests as the delta between the actual output and minReturn being captured/transferred by a third party, rather than an unconditional draining of the entire principal. Similar "capturing the spread within the slippage band" outcomes may also occur under MEV/sandwich behaviors in open markets.Therefore, we believe the practical impact is more accurately categorized as **Medium** (bounded by minReturn within a single transaction).*

*Regarding **Likelihood**: Code Accessibility vs. Real-World Exploitation Prerequisites*

While the mechanism can be demonstrated in isolation, successful exploitation in production depends on several practical prerequisites that materially reduce probability:

- *Production barriers are integral to risk assessment: In our current production environment, the set of adapters used by DexRouter is deployed and maintained by OKXLabs and used by OKX under a whitelist/controlled-set model. Arbitrary third-party adapter injection is not supported. Therefore, the audit example path using MaliciousRFQAdapter is not equivalent to a standard production-reachable path.*
- *Request for production-bounded validation: To evaluate actual production reachability, we recommend providing a PoC that reproduces the issue without using MaliciousRFQAdapter (or any other malicious custom adapter), relying only on the adapter combinations currently available through the router.*

- *High Likelihood threshold not met (per your matrix): The attack typically requires multiple conditions to align simultaneously (e.g., callback-enabled adapters in the route, a malicious/controllable counterparty callback behavior, and a specific hop composition and timing window). The contracts are open-source and have been public for a notable period. To date, we have not observed real attacks exploiting this path in production (this does not prove absence of risk, but serves as a supporting data point on observed incidence). Also, we have not seen objective evidence supporting that this scenario meets the audit matrix threshold for High Likelihood (50–60% annual probability).*

  *Considering these operational constraints and prerequisites, we propose the likelihood is more appropriately assessed as **Low**.*

  Seeking Consensus on Final Severity

  We respectfully request a final severity rating that balances theoretical soundness with practical context:
- *We will consider targeted defense-in-depth measures to further harden the execution flow.*
- *We assert that current production controls (a controlled adapter set and a controlled integration surface) significantly reduce real-world exploitability.*
- *We propose a final severity of **Low/Medium**, based on Medium Impact * Low Likelihood.*

MixBytes:

We agree to classify this issue as Medium severity, based on a Medium Impact and Medium Likelihood assessment.

The described behavior can allow third parties to capture value during cross-contract callbacks when direct adapter-to-adapter transfers are used. The resulting loss is bounded by slippage constraints and manifests within a single transaction, rather than enabling unrestricted draining of user funds or protocol assets.

Exploitation depends on specific routing compositions and callback-capable counterparties, which limits frequency but does not eliminate plausibility in a composable execution environment. Given these factors, a medium likelihood classification is appropriate.

We note that this issue remains distinct in mechanism from other reentrancy findings, and recommended design hardening measures remain relevant to reduce residual execution risk.

Client:

We acknowledge the theoretical existence of this mechanism-level risk in callback-heavy, composable routing environments. Our primary goal in this discussion is to align on a disposition that accurately reflects the practical risk profile in our production context and the proportionality of the proposed remediation.We understand your framework evaluates worst-case scenarios from a code perspective. However, industry best practices in risk assessment also consider (i) whether the user-visible impact is already bounded by existing protections, and (ii) whether the recommended mitigation is proportionate and practical to deploy without introducing significant regressions (e.g., gas costs and engineering complexity).

Regarding Impact: Theoretical Capability vs. Exploitable Scope

Under supported routing assumptions, minReturn bounds the user-visible outcome of the swap. As a result, even if adversarial behavior is introduced via external callbacks, the maximum user loss is constrained within the configured slippage tolerance, rather than enabling an unbounded protocol-level drain.

Regarding Remediation Practicality: Proportionality and Deployment Feasibility

We acknowledge the proposed mitigation ("always route via DexRouter as the intermediate holder between all steps"). However, this would be a major architectural change that materially increases gas usage and engineering complexity across a broad set of routes. At this stage, we do not consider it practical for production deployment without significant performance and integration trade-offs.

Current Mitigations / Hardening:

As a primary hardening measure against callback-based reentrancy patterns, we have already added nonReentrant protection on external router entry points, which materially reduces the exploitability of reentrancy-driven manipulations in supported flows.

Request:

Given the bounded user-visible impact under minReturn and the disproportionate cost of the proposed architectural change, we respectfully request removing it from the Findings list, rather than treating it as a reportable vulnerability.

MixBytes:

We appreciate the additional context and the focus on proportionality and production practicality. We agree that enforcing the router as the intermediate holder for every step could increase gas costs and introduce engineering trade-offs across a wide range of routes. That said, addressing the core risk does not necessarily require a major architectural redesign: the minimal change is to disable the "direct adapter - adapter transfer" optimization so that each adapter step always receives funds via an explicit transfer

from router.

An example of a small patch to remove the `noTransfer` mode entirely.

```
// Option A (simplest): remove `noTransfer` optimization entirely.
// Only the relevant bits are shown.
function _exeHop(...) private {
    // ...
    for (uint256 i = 0; i < hopLength; ++i) {
        // compute `to` as before (router / receiver / next adapter)
        _exeForks(payer, refundTo, to, batchAmount, hops[i]); // no noTransfer
        // noTransfer logic removed
    }
}

function _exeForks(
    address payer,
    address refundTo,
    address to,
    uint256 batchAmount,
    RouterPath memory path
) private {
    for (uint256 i = 0; i < path.mixAdapters.length; i++) {
        // decode weight/pool/reverse as before...

        // Always fund the adapter step explicitly (no direct adapter->adapter pass-through)
        uint256 amt = (weight == 10_000) ? batchAmount : (batchAmount * weight) / 10_000;
        _transferInternal(payer, path.assetTo[i], path.fromToken, amt);

        _exeAdapter(...);
    }
}
```

If maintaining this optimization is important for performance, an alternative is to keep it behind a configuration flag encoded into user transaction parameters (defaulting to disabled, if not provided), so it can be selectively enabled where the risk profile is acceptable. However, this approach would require changing the decoding mechanisms of user swap intents.

```
function _exeHop(..., bool direct /* default as false if not provided */ ) private {
    // ...
    for (uint256 i = 0; i < hopLength; ++i) {
        bool toNext = (i < hopLength - 1 && hops[i + 1].assetTo.length == 1);
        _exeForks(
            payer,
            refundTo,
            to,
            batchAmount,
            hops[i],
            /*noTransfer*/ (direct && toNext)
        );
    }
}

function _exeForks(..., bool noTransfer) private {
    // ...
    if (!noTransfer) {
        _transferInternal(...); // fund step explicitly
    }
    _exeAdapter(...);
}
```

Also, the risk surface is not limited to "callback-capable adapters" (e.g., RFQ maker callbacks). Tokens with transfer hooks (e.g., AMPL, which is ERC-777) can introduce arbitrary callbacks during transfers; in those scenarios, `nonReentrant` on external router entry points is not a complete mitigation for the underlying "funds temporarily living outside the router" pattern.
*Given the **Medium** severity, we are comfortable treating this as non-urgent. If you prefer not to implement the mitigation, we can mark the finding as Acknowledged while keeping the rationale documented for completeness, so it's clear the team reviewed the concern, even if it does not require urgent remediation.*

***Client**:*
We appreciate the thorough analysis and constructive engagement from you throughout this discussion. We would like to provide our final input, beginning with our remediation decision before addressing the severity classification.
Remediation: We Accept the Recommended Fix
We have decided to adopt your recommended approach and remove the noTransfer optimization in _exeHop/_exeForks entirely.
Severity: We Request Low
While we are implementing the fix regardless of severity, we respectfully maintain that this finding does not reach Medium under your risk classification framework.
Impact:
The economic outcome of this mechanism is indistinguishable from standard MEV/sandwich extraction that occurs in every DEX trade across every protocol. Sandwich bots on Uniswap, 1inch, Paraswap, Curve, and all other DEX aggregators routinely extract exactly the same slippage-bounded delta from users. This is not a protocol-specific vulnerability — it is a baseline condition of on-chain trading that all DEX users accept when they set a slippage tolerance.
If slippage-bounded value extraction constitutes Medium Impact, then by the same standard, every DEX and DEX aggregator in production carries a permanent Medium-Impact vulnerability — which would render the severity distinction meaningless. We believe the appropriate classification for risk that is economically equivalent to market-wide MEV exposure is Low Impact — or, more precisely, that it falls outside the defined Impact categories and should default to the lowest applicable level.

Likelihood: Below the Medium Threshold

For an external actor to exploit this, multiple independent prerequisites must align simultaneously:

1. *The adapter set deployed for production use is curated and does not include arbitrary third-party adapters. We have previously requested for a production-feasible PoC using only adapter combinations currently available through the router.*
2. *The callback counterparty must be adversarial, aware of the routing composition in real time, and able to execute extraction (e.g., skim()) within the callback window — while the net economic payoff (slippage delta minus gas costs) remains positive.*

   The simultaneous alignment of these conditions places the probability well below the 10–20% annual threshold defined for Medium Likelihood. We believe a Likelihood assessment should be grounded in demonstrated exploitability under realistic conditions, not in theoretical vectors that do not map to the actual code path. Combined with zero observed incidents despite the contracts being public for a notable period, we believe this finding falls below the Medium Likelihood threshold.

   Even accepting Medium Impact (which we dispute), Low Likelihood yields Low severity. Under full assessment (Low Impact × Low Likelihood), the result is also ***Low****. We look forward to your response and are confident we can reach a fair consensus that reflects both theoretical rigor and production reality.*

MixBytes:

We agree to classify this issue as Low severity.

L-17	Reentrancy Guard Misconfiguration Causes Wrapper Reverts		
Severity	Low	**Status**	Fixed in 2498b86a

Description

In DexRouter, multiple external nonReentrant wrapper functions invoke public nonReentrant functions.

Under OpenZeppelin ReentrancyGuard, this nested guarded execution reverts, rendering the wrappers non-functional.

Affected call paths:

- unxswapByOrderId(...) -> unxswapTo(...)
- smartSwapByInvest(...) -> smartSwapByInvestWithRefund(...)
- dagSwapByOrderId(...) -> dagSwapTo(...)

Recommendation

We recommend removing nonReentrant from the listed external wrapper functions, as the corresponding public functions are already protected.

Client's Commentary

Accept. Fixed in: PR-25

3. About MixBytes

MixBytes is a leading provider of smart contract auditing and blockchain security research, helping Web3 projects enhance their security and reliability.

Since its inception, MixBytes has been dedicated to safeguarding innovation in **DeFi** through rigorous audits and cutting-edge technical research.

Our team brings together experienced engineers, security auditors, and blockchain researchers with deep expertise in smart contract security and protocol design.

With years of proven experience in Web3, MixBytes combines in-depth technical excellence with a proactive, security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by leading blockchain protocols such as **Lido**, **Aave**, **Curve**, **1inch**, **Fluid**, **Gearbox**,**Resolv**, and others. MixBytes has successfully secured billions of dollars in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems, backed by years of hands-on experience.
- **Innovative Research:** MixBytes actively contributes to blockchain security research and open-source tools, sharing insights with the global community.

Our Services

- **Smart Contract Audits:** Comprehensive security assessments of DeFi protocols to detect and mitigate vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical studies, economic and protocol-level modeling for Web3 projects.
- **Custom Security Solutions:** Tailored frameworks and advisory for complex decentralized systems and blockchain ecosystems.

Contact Information

https://mixbytes.io/

https://github.com/mixbytes/audits_public

https://x.com/MixBytes

hello@mixbytes.io